claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_793a5d86-1c4\agent-a3562776cc10bf99b.jsonl`
- SHA-256: `d3f195e79e26e6e6f7e1ec18f1a46c68a96f3df8d4ad8e6f51f301d23c19a2dc`
- Source modified: `2026-07-02T08:17:59+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_793a5d86-1c4`
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`

Transcript

1. user (2026-07-02T08:17:19.097Z)

You are an ADVERSARIAL verifier. Independently re-check the fate verdict for OPEN PR #416 ("fix(storage): gdrive-api chunked streaming download + safe idempotent put (R1-18)", branch origin/fix/gdrive-api-chunked-download-safe-put) on Khelsutra/badminton-highlight-indexer. Try to REFUTE the analyst.

Analyst's verdict: recommended_fate=TAKE_OVER_CLEAN; superseded_by=none (confidence n/a).
Analyst's master_evidence: Master @ d00b4db backend/storage/gdrive_api.py is UNCHANGED from #338 baseline for both hazards the PR fixes. (1) Whole-file buffering: line 144 `self._service.files().get_media(fileId=file_id).execute()` materializes the entire object as one bytes value, then lines 147-148 `with io.open(dst,'wb') as fh: fh.write(data)` write straight to dst ? no chunked MediaIoBaseDownload, no `_downloader` seam, no `_DOWNLOAD_CHUNK_BYTES`, no `.part`/os.replace anywhere (git grep confirms none of those tokens exist in the file). (2) Delete-before-upload data-loss window: put() lines 160-170 still do `existing = self._resolve_id(ref.key); if existing: self._service.files().delete(fileId=existing).execute()` (line 165) FOLLOWED BY `self._service.files().create(...)` (lines 166-170) ? deletes first, then creates; no `files().update` in master. Docstring lines 21-23 still say put 'deletes an existing file at the same path before creating'. File history (git log) shows gdrive_api.py last touched only by #338 (feat), #360 (isort), #387 (ruff format) ? none of the merged #422-#428 touched it; tests/test_storage_gdrive_api.py has 10 test fns, none of the PR's 4 new streaming/overwrite-safety tests present.
Analyst's conflict_nature: clean ? mergeable=MERGEABLE. PR touches only backend/storage/gdrive_api.py and tests/test_storage_gdrive_api.py, neither of which was modified by any merged PR since the PR's base; the diff hunks (fetch_to_local body, put() body, docstring, new _downloader/_DOWNLOAD_CHUNK_BYTES, new tests) apply to unchanged master regions. No adjacent-edit or already-done conflict.
Analyst's rationale: Master still whole-file buffers on download (get_media().execute(), gdrive_api.py:144) and still deletes-before-create in put() (delete at :165 then create at :166), exactly the two hazards this PR fixes ? the intent is entirely absent from master and NOT superseded by any of #422-#428 (none touched this file). PR is MERGEABLE with all CI green, applies cleanly to current master, is well-tested (4 new offline tests, +correct .part/os.replace and update-in-place logic), and preserves the existing StorageBackend/idempotence contract with no caller-facing behavior change. Adopt and merge as-is after a local full-suite/lint/type re-verify per the pre-LGTM gate.

Already MERGED onto master (verify against actual master code, do not assume):
- #422 = fix for --output-dir clobbering config.json output.output_dir (CODE_MAP updated for it)
- #423 = docs: health-remediation plan added + DOC_STATUS/README/NEXT_STEPS registered; audit marked snapshot
- #424 = P2: /api/process idempotent by content hash (#340 re-bill fix)
- #425 = P3: /api/source resolves remote source into output-scoped source_cache, reuses across Range requests, warms annotation proxy
- #426 = P4: bounded ffprobe/ffmpeg subprocess timeouts on serving paths

```
- #427 = P5: clean remote-input scratch dirs after process/compile/assets/CLI; durable reports
- #428 = P1/P6: backend/storage/gcs.py lazy-import mypy typing fix
master HEAD = d00b4db "fix gcs storage lazy import typing (#428)"

Working dir: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer on master.
SHARED checkout ? READ-ONLY, no branch checkout, no working-tree changes.

Your job ? verify by looking at code yourself, do NOT take the analyst's word:
- If the verdict is CLOSE_SUPERSEDED: the risk is closing a PR whose work is NOT actually on
master (losing real work). Open the specific master file:line the analyst cited AND the PR diff
(gh pr diff 416 -R Khelsutra/badminton-highlight-indexer). Confirm master truly implements the
SAME intent (not just a similarly-named symbol). If the PR does anything master lacks (extra
tests, a stricter guard, a different file), flag it ? that means NOT fully superseded.
- If the verdict is TAKE_OVER_*: the risk is that the work IS already on master (would be
redundant/conflicting) OR the change is buggy. Confirm master still lacks it, and sanity-check
the change is correct.
- Default to skepticism: only agree=true if the evidence in code clearly supports it.

Return ONLY the structured object.
```

2. assistant (2026-07-02T08:17:21.969Z)

I'll independently verify the analyst's claims by examining both the PR diff and the actual master code.

3. user (2026-07-02T08:17:24.621Z)

```
d00b4db fix gcs storage lazy import typing (#428)
---
b6214e5 chore: ruff format the whole repo (259 files) (#387)
f523d3a refactor: enable isort (I) in ruff, auto-fix 143 files (Q6) (#360)
819e395 feat(serving): M12 ? gdrive-api:// OAuth/API StorageBackend (D14 Northstar) (#338)
```

4. user (2026-07-02T08:17:26.281Z)

```
diff --git a/backend/storage/gdrive_api.py b/backend/storage/gdrive_api.py
index 210edce..3252e39 100644
--- a/backend/storage/gdrive_api.py
+++ b/backend/storage/gdrive_api.py
@@ -18,9 +18,12 @@

 **Scope caveats (M12):** handles **binary** files (videos / reels / JSON) ? Google-native Docs
 (Sheets/Slides) have no direct download and would need ``files().export_media`` (out of scope;
export
-them first). ``put`` is idempotent (deletes an existing file at the same path before creating).
Path
-resolution keys on ``(name, parent)``; if Drive already holds duplicate-named files under one
folder
-(a rare pre-existing state this backend never creates), resolution returns the first match.
+them first). ``put`` is idempotent: an existing file at the same path is overwritten **in
place** via
+``files().update`` (same file id; the old bytes survive until the new upload succeeds ? never
+delete-then-create). Path resolution keys on ``(name, parent)``; if Drive already holds
+duplicate-named files under one folder (a rare pre-existing state this backend never creates),
+resolution returns the first match. Downloads stream in chunks (``MediaIoBaseDownload``) ? a
+multi-GB reel is never buffered whole in RAM.
 """

 from __future__ import annotations
@@ -35,6 +38,9 @@

 _FOLDER_MIME = "application/vnd.google-apps.folder"
 _DRIVE_SCOPE = "https://www.googleapis.com/auth/drive"
+# Download chunk size: bounds fetch_to_local RAM no matter the object size (vs. the
```

```
googleapiclient
+# default of 100 MiB) while keeping the number of round-trips per multi-GB reel reasonable.
+_DOWNLOAD_CHUNK_BYTES = 32 * 1024 * 1024


 def _q_escape(name: str) -> str:
@@ -83,6 +89,18 @@ def _media_body(self, local_path: str, content_type: Optional[str]) -> Any:
             local_path, mimetype=content_type, resumable=True
         )  # pragma: no cover

+    def _downloader(self, fh: Any, request: Any) -> Any:
+        """The chunked-download driver for a ``files().get_media`` request ? lazy
+        ``MediaIoBaseDownload`` (real path). A test seam: monkeypatch this to avoid the
+        ``googleapiclient`` dependency (the fakes drive the same ``next_chunk()`` loop)."""
+        from googleapiclient.http import (
+            MediaIoBaseDownload,  # type: ignore  # pragma: no cover
+        )
+
+        return MediaIoBaseDownload(
+            fh, request, chunksize=_DOWNLOAD_CHUNK_BYTES
+        )  # pragma: no cover
+
     # -- path -> id resolution
-------------------------------------------------------------------
     def _resolve_id(self, key: str, *, create_dirs: bool = False) -> Optional[str]:
         """Walk ``key``'s path components from the root folder, returning the leaf's file id
(or
@@ -140,12 +158,24 @@ def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
         file_id = self._resolve_id(ref.key)
         if not file_id:
             raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
-        data = (
-            self._service.files().get_media(fileId=file_id).execute()
-        )  # alt=media ? raw bytes
+        request = self._service.files().get_media(fileId=file_id)  # alt=media ? bytes
         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
-        with io.open(dst, "wb") as fh:
-            fh.write(data)
+        # Stream in bounded chunks to a ``.part`` sibling, then rename into place: a multi-GB
+        # reel never sits whole in RAM (the old ``get_media().execute()`` buffered the entire
+        # object as one bytes value ? OOM fodder on a memory-limited Cloud Run box), and a
+        # mid-download failure can't leave a truncated ``dst`` that a later ``overwrite=False``
+        # call would trust as complete.
+        tmp = dst + ".part"
+        try:
+            with io.open(tmp, "wb") as fh:
+                downloader = self._downloader(fh, request)
+                done = False
+                while not done:
+                    _, done = downloader.next_chunk()
+            os.replace(tmp, dst)
+        finally:
+            if os.path.exists(tmp):
+                os.remove(tmp)
         return dst

     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
@@ -158,16 +188,22 @@ def put(self, local_path, ref, *, content_type=None) -> StorageRef:
         if parent_id is None:
             raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")
         # Idempotent: Drive allows duplicate (name, parent), so a plain create() on a re-put
would
-        # ORPHAN the prior file (and reads would keep returning the stale first match). Delete
an
-        # existing file at this exact path first, so put() overwrites rather than accumulates.
```

```
+        # ORPHAN the prior file (and reads would keep returning the stale first match).
Overwrite
+        # an existing file at this exact path IN PLACE via files().update ? the file id is kept
+        # and the OLD bytes survive until the NEW upload succeeds. (The previous delete-then-
create
+        # ordering lost the object outright when the create failed mid-upload; create-first-
then-
+        # delete would instead open a duplicate-(name, parent) window where reads could resolve
+        # either copy.)
+        media = self._media_body(str(local_path), content_type)
         existing = self._resolve_id(ref.key)
         if existing:
-            self._service.files().delete(fileId=existing).execute()
-        self._service.files().create(
-            body={"name": name, "parents": [parent_id]},
-            media_body=self._media_body(str(local_path), content_type),
-            fields="id",
-        ).execute()
+            self._service.files().update(fileId=existing, media_body=media).execute()
+        else:
+            self._service.files().create(
+                body={"name": name, "parents": [parent_id]},
+                media_body=media,
+                fields="id",
+            ).execute()
         return ref

     def exists(self, ref) -> bool:
diff --git a/tests/test_storage_gdrive_api.py b/tests/test_storage_gdrive_api.py
index c89af22..44b001f 100644
--- a/tests/test_storage_gdrive_api.py
+++ b/tests/test_storage_gdrive_api.py
@@ -2,8 +2,10 @@

 Fully offline: a tiny in-memory ``_FakeDrive`` stands in for the googleapiclient Drive v3
``service``
 (injected, mirroring the GCS fake-client tests). No ``google-api-python-client`` / network
needed.
-Asserts: scheme parsing, registry wiring, the hyphen?underscore settings lookup, and the full
-path-keyed round-trip (put ? exists ? stat ? fetch ? list ? delete).
+Asserts: scheme parsing, registry wiring, the hyphen?underscore settings lookup, the full
+path-keyed round-trip (put ? exists ? stat ? fetch ? list ? delete), the chunked (streaming)
+download path, and overwrite safety: a re-put updates in place (same file id) and a mid-upload
+failure leaves the OLD object intact ? never delete-before-upload.
 """

 import re
@@ -26,6 +28,17 @@ def execute(self):
         return self._value


+class _RaiseExec:
+    """An operation whose ``execute()`` fails (simulated network/quota mid-upload error) ?
+    crucially WITHOUT having mutated the fake Drive first."""
+
+    def __init__(self, msg):
+        self._msg = msg
+
+    def execute(self):
+        raise RuntimeError(self._msg)
+
+
 class _FakeFiles:
     def __init__(self, drive):
         self.d = drive
```

```
@@ -46,6 +59,8 @@ def list(self, q="", spaces=None, fields=None, pageToken=None):
         return _Exec({"files": out})  # one page, no nextPageToken

     def create(self, body=None, media_body=None, fields=None):
+        if media_body is not None and self.d.fail_uploads:
+            return _RaiseExec("simulated mid-upload failure (network/quota)")
         nid = self.d.new_id()
         self.d.nodes[nid] = {
             "name": body["name"],
@@ -59,6 +74,13 @@ def create(self, body=None, media_body=None, fields=None):
                 self.d.content[nid] = fh.read()
         return _Exec({"id": nid})

+    def update(self, fileId=None, media_body=None, fields=None):
+        if self.d.fail_uploads:
+            return _RaiseExec("simulated mid-upload failure (network/quota)")
+        with open(media_body, "rb") as fh:  # _media_body patched ? the local path
+            self.d.content[fileId] = fh.read()
+        return _Exec({"id": fileId})
+
     def get_media(self, fileId=None):
         return _Exec(self.d.content.get(fileId, b""))

@@ -85,6 +107,9 @@ def __init__(self):
         self.nodes = {"root": {"name": "root", "mimeType": "folder", "parents": []}}
         self.content = {}
         self._n = 0
+        self.fail_uploads = (
+            False  # flip on to make the NEXT upload (create/update) fail
+        )

     def new_id(self):
         self._n += 1
@@ -94,11 +119,36 @@ def files(self):
         return _FakeFiles(self)


+class _FakeDownloader:
+    """Stands in for ``googleapiclient.http.MediaIoBaseDownload``: drains the fake
``get_media``
+    request's bytes into the file handle a few bytes per ``next_chunk()`` call, so the
+    production streaming loop is exercised chunk by chunk (never one whole-file buffer)."""
+
+    chunk_size = 4
+
+    def __init__(self, fh, request):
+        self._fh = fh
+        self._data = request.execute()  # the fake request; a real one streams over HTTP
+        self._pos = 0
+        self.chunks = 0
+
+    def next_chunk(self):
+        piece = self._data[self._pos : self._pos + self.chunk_size]
+        self._fh.write(piece)
+        self._pos += len(piece)
+        self.chunks += 1
+        return None, self._pos >= len(self._data)
+
+
 @pytest.fixture
 def be(monkeypatch):
-    """A GDriveApiStorage on a fake service, with _media_body patched to the local path (no
-    googleapiclient dependency ? the fake `create` reads the bytes from that path)."""
+    """A GDriveApiStorage on a fake service, with the two googleapiclient seams patched:
+    _media_body ? the local path (the fake `create`/`update` read the bytes from it) and
```

```
+    _downloader ? _FakeDownloader (drives the real next_chunk() streaming loop)."""
     monkeypatch.setattr(GDriveApiStorage, "_media_body", lambda self, p, ct: p)
+    monkeypatch.setattr(
+        GDriveApiStorage, "_downloader", lambda self, fh, req: _FakeDownloader(fh, req)
+    )
     return GDriveApiStorage(service=_FakeDrive())


@@ -163,7 +213,7 @@ def test_roundtrip_put_exists_stat_fetch_list_delete(be, tmp_path):


 def test_put_is_idempotent_no_duplicate_on_reput(be, tmp_path):
-    """Re-putting the same path OVERWRITES (delete-then-create) ? Drive allows duplicate names,
so a
+    """Re-putting the same path OVERWRITES (update-in-place) ? Drive allows duplicate names, so
a
     naive create would orphan the old file and reads would keep returning the stale one."""
     ref = parse_uri("gdrive-api://Coaching/clip.mp4")
     v1 = tmp_path / "v1.mp4"
@@ -181,6 +231,98 @@ def test_put_is_idempotent_no_duplicate_on_reput(be, tmp_path):
     )  # the latest bytes, not the orphaned first


+def test_reput_updates_in_place_preserving_file_id(be, tmp_path):
+    """The idempotent overwrite is files().update on the EXISTING id ? the Drive file id is
+    stable across re-puts (shared links / ids held elsewhere keep working)."""
+    ref = parse_uri("gdrive-api://Coaching/clip.mp4")
+    v1 = tmp_path / "v1.mp4"
+    v1.write_bytes(b"FIRST")
+    v2 = tmp_path / "v2.mp4"
+    v2.write_bytes(b"SECOND")
+    be.put(str(v1), ref)
+    id_before = be._resolve_id(ref.key)
+    be.put(str(v2), ref)
+    assert be._resolve_id(ref.key) == id_before
+
+
+def test_put_failure_mid_upload_keeps_old_object_intact(be, tmp_path):
+    """A re-put whose upload fails must leave the PREVIOUS object readable. The old
+    delete-then-create ordering deleted first, so a failed create lost the reel at both ends;
+    update-in-place only replaces content once the upload succeeds."""
+    ref = parse_uri("gdrive-api://Coaching/reel.mp4")
+    v1 = tmp_path / "v1.mp4"
+    v1.write_bytes(b"OLD-REEL")
+    v2 = tmp_path / "v2.mp4"
+    v2.write_bytes(b"NEW-REEL")
+    be.put(str(v1), ref)
+
+    be._service.fail_uploads = True
+    with pytest.raises(RuntimeError, match="mid-upload"):
+        be.put(str(v2), ref)
+
+    assert be.exists(ref)  # still there ?
+    dst = tmp_path / "out.mp4"
+    be.fetch_to_local(ref, str(dst))
+    assert dst.read_bytes() == b"OLD-REEL"  # ? with the old bytes intact
+
+    be._service.fail_uploads = False  # and a retry succeeds: still exactly one file
+    be.put(str(v2), ref)
+    listed = [r.uri for r in be.list(parse_uri("gdrive-api://Coaching"))]
+    assert listed == ["gdrive-api://Coaching/reel.mp4"]
+    dst2 = tmp_path / "out2.mp4"
+    be.fetch_to_local(ref, str(dst2))
+    assert dst2.read_bytes() == b"NEW-REEL"
+
```

```
+
+def test_fetch_large_file_streams_in_chunks(be, tmp_path, monkeypatch):
+    """A file much larger than the chunk size is drained via MANY next_chunk() calls ? the
+    production loop streams; it never materializes the object as one .execute() buffer."""
+    data = b"0123456789" * 5  # 50 bytes ? the fake 4-byte chunk
+    src = tmp_path / "big.mp4"
+    src.write_bytes(data)
+    ref = parse_uri("gdrive-api://Coaching/big.mp4")
+    be.put(str(src), ref)
+
+    made = []
+
+    def spy(self, fh, req):
+        d = _FakeDownloader(fh, req)
+        made.append(d)
+        return d
+
+    monkeypatch.setattr(GDriveApiStorage, "_downloader", spy)
+    dst = tmp_path / "out" / "big.mp4"
+    be.fetch_to_local(ref, str(dst))
+    assert dst.read_bytes() == data  # byte-identical after reassembly
+    assert made[0].chunks == 13  # ceil(50 / 4) ? chunked, not one whole-file buffer
+    assert not (tmp_path / "out" / "big.mp4.part").exists()  # temp renamed away
+
+
+def test_fetch_failure_mid_download_leaves_no_truncated_dst(be, tmp_path, monkeypatch):
+    """If the stream dies mid-download, dst must NOT exist (a truncated file would be trusted
+    as complete by a later overwrite=False call) and the .part temp is cleaned up."""
+
+    class _Boom(_FakeDownloader):
+        def next_chunk(self):
+            if self.chunks >= 2:
+                raise RuntimeError("simulated mid-download network failure")
+            return super().next_chunk()
+
+    src = tmp_path / "big.mp4"
+    src.write_bytes(b"0123456789ABCDEF")
+    ref = parse_uri("gdrive-api://Coaching/big.mp4")
+    be.put(str(src), ref)
+
+    monkeypatch.setattr(
+        GDriveApiStorage, "_downloader", lambda self, fh, req: _Boom(fh, req)
+    )
+    dst = tmp_path / "out.mp4"
+    with pytest.raises(RuntimeError, match="mid-download"):
+        be.fetch_to_local(ref, str(dst))
+    assert not dst.exists()
+    assert not (tmp_path / "out.mp4.part").exists()
+
+
 def test_fetch_missing_raises(be, tmp_path):
     with pytest.raises(FileNotFoundError):
         be.fetch_to_local(
```

5. assistant (2026-07-02T08:17:33.345Z)

Master and PR diff are clear. Let me verify the actual master code at the cited lines to confirm the hazards are present and unfixed.

6. user (2026-07-02T08:17:36.351Z)

```
        if os.path.exists(dst) and not overwrite:
            return dst
        file_id = self._resolve_id(ref.key)
        if not file_id:
```

```
            raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
        data = (
            self._service.files().get_media(fileId=file_id).execute()
        )  # alt=media ? raw bytes
        os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
        with io.open(dst, "wb") as fh:
            fh.write(data)
        return dst

    def put(self, local_path, ref, *, content_type=None) -> StorageRef:
        parent_path, name = self._split(ref.key)
        parent_id = (
            self._resolve_id(parent_path, create_dirs=True)
            if parent_path
            else self._root_id
        )
        if parent_id is None:
            raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")
        # Idempotent: Drive allows duplicate (name, parent), so a plain create() on a re-put
would
        # ORPHAN the prior file (and reads would keep returning the stale first match). Delete
an
        # existing file at this exact path first, so put() overwrites rather than accumulates.
        existing = self._resolve_id(ref.key)
        if existing:
            self._service.files().delete(fileId=existing).execute()
        self._service.files().create(
            body={"name": name, "parents": [parent_id]},
            media_body=self._media_body(str(local_path), content_type),
            fields="id",
        ).execute()
        return ref

    def exists(self, ref) -> bool:
        return self._resolve_id(ref.key) is not None

    def stat(self, ref) -> StorageStat:
        file_id = self._resolve_id(ref.key)
        if not file_id:
            raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
        meta = (
```

7. user (2026-07-02T08:17:37.833Z)

```
EXIT=1
```

8. assistant (2026-07-02T08:17:41.163Z)

```
Both hazards are confirmed present in master, and none of the PR's new tokens exist. Let me
confirm PR mergeability/CI and that no merged PR touched these two files since the base.
```

9. user (2026-07-02T08:17:44.015Z)


```
{"baseRefName":"master","files":[{"path":"backend/storage/gdrive_api.py","additions":52,"deletio
ns":16,"changeType":"MODIFIED"},{"path":"tests/test_storage_gdrive_api.py","additions":147,"dele
tions":5,"changeType":"MODIFIED"}],"headRefOid":"6afacdd8e3974200391da65da16cdcda3ee7df39","merg
eStateStatus":"CLEAN","mergeable":"MERGEABLE"}
```


10. user (2026-07-02T08:17:44.673Z)

```
b6214e5 chore: ruff format the whole repo (259 files) (#387)
f523d3a refactor: enable isort (I) in ruff, auto-fix 143 files (Q6) (#360)
819e395 feat(serving): M12 ? gdrive-api:// OAuth/API StorageBackend (D14 Northstar) (#338)
```

11. assistant (2026-07-02T08:17:53.642Z)

All claims verified independently:

1. **Whole-file buffering hazard present in master** (gdrive_api.py:143-148): `get_media(fileId=file_id).execute()` returns raw bytes, then `io.open(dst, "wb")` writes the whole buffer. No chunking.

2. **Delete-before-create data-loss window present in master** (put(), lines ~165-171): `existing = self._resolve_id(ref.key); if existing: files().delete(...).execute()` followed by `files().create(...)`. Deletes first, then creates.

3. **None of the PR's new tokens exist in master** ? git grep for `_DOWNLOAD_CHUNK_BYTES|MediaIoBaseDownload|_downloader|.part|os.replace|files().update` returns EXIT=1 (no matches).

4. **No merged PR since base touched these files** ? both files' last commits are #387/#360/#338; none of #422-#428 appear.

5. **PR is MERGEABLE / CLEAN**, base=master, touches only the two files (both untouched by merged PRs).

6. **Change is correct** ? fetch streams via `next_chunk()` to `dst + ".part"` then `os.replace`, with a `finally` cleanup; put uses `files().update` on existing id (old bytes survive until new upload succeeds) else `create`. The 4 new offline tests exercise streaming, in-place-update id preservation, mid-upload failure preserving old bytes, and mid-download failure leaving no truncated dst.

The analyst's verdict (TAKE_OVER_CLEAN, not superseded) is fully supported by the code. I could not refute it.

12. user (2026-07-02T08:17:59.522Z)

Structured output provided successfully